

ESP32-C6 WiFi Daughterboard (external XIAO C6 over SPI)

Status: current production wiring for the ESP32-P4 tier.

Why it exists: the CrowPanel's **onboard** ESP32-C6 talks to the P4 over **SDIO**, and that SDIO bus shares the P4's **SDMMC controller** with the **microSD card**. Using the onboard C6 for WiFi therefore blocks the SD card (and vice-versa). The workaround is to leave the onboard C6 unused and add an **external Sseed XIAO ESP32-C6** on the P4's **SPI** bus (J7 wireless-module header). SPI frees the SDMMC controller, so **WiFi and the SD card work at the same time**.

Background — the pin conflict

The Elecrow **CrowPanel Advance ESP32-P4** has:

- An **onboard ESP32-C6** co-processor for WiFi, wired to the P4 over **SDIO** (the P4 itself has no native WiFi radio).
- A **microSD card** slot wired to the P4's **SDMMC** peripheral.

The SDIO transport to the onboard C6 and the SD card both need the SDMMC controller / the same signal pins. You can have WiFi **or** the SD card on that bus, but not both reliably — bringing up esp-hosted over SDIO left the SD card unable to mount (`sdmmc_card_init failed`).

Because the firmware needs the SD card for firmware-update images (`/esp32_firmware.bin` , `/stm32_firmware.bin`) **and** WiFi for the Flask dashboard / wireless flashing, both are required simultaneously.

The workaround — external XIAO C6 on SPI

WiFi is provided instead by an **external Sseed XIAO ESP32-C6** running the **esp-hosted SPI slave** firmware, connected to the P4's **J7 wireless-module header** over **SPI**. The onboard C6/SDIO path is no longer used for WiFi, which leaves the SDMMC controller free for the SD card.

The firmware side is unchanged from the app's point of view: the standard `esp_wifi` API in `wifi_bridge.cpp` is transparently forwarded to the XIAO C6 by `esp_hosted` / `esp_wifi_remote` (declared in `ESP32P4/main/idf_component.yml` (`../ESP32P4/main/idf_component.yml`)). Only the transport (SPI vs SDIO) and its pins changed, in `ESP32P4/sdkconfig.defaults` (`../ESP32P4/sdkconfig.defaults`).

Wiring (P4 J7 header → XIAO C6)

Signal	P4 GPIO	XIAO pin	XIAO GPIO	sdkconfig symbol
MOSI	48	D10	18	CON- FIG_ESP_HOSTED_ SPI_HSPI_GPIO_M OSI
MISO	47	D9	20	CON- FIG_ESP_HOSTED_ SPI_HSPI_GPIO_M ISO
CLK	26	D8	19	CON- FIG_ESP_HOSTED_ SPI_HSPI_GPIO_C LK
CS	29	D2	2	CON- FIG_ESP_HOSTED_ SPI_HSPI_GPIO_C S
Handshake	30	D1	1	CON- FIG_ESP_HOSTED_ SPI_GPIO_HANDSH AKE
Data Ready	31	D7	17	CON- FIG_ESP_HOSTED_ SPI_GPIO_DATA_R EADY
Reset/EN	32	RST/EN	—	CON- FIG_ESP_HOSTED_ SPI_GPIO_RESET_ SLAVE (active- LOW)
Power	3V3	3V3	—	—
Ground	GND	GND	—	—

Notes:

- **Reset pin GPIO32** is the pin freed up when the STM32 `B00T0` / `NRST` self-test was removed (STM32 updates now go through the Katapult bootloader, so the P4 no longer needs to drive STM32 reset/boot pins). Reset is **active-low** (the XIAO

EN pin: LOW = held in reset, HIGH = running).

- **Do NOT connect XIAO D4/D5 (GPIO22/23)** — those are physically tied to the P4's I²C bus (GT911 touch + backlight controller).
- Use short jumpers (< 10 cm) on the SPI lines to keep the 10 MHz clock clean.

Key config settings (`ESP32P4/sdkconfig.defaults`)

```
CONFIG_ESP_HOSTED_ENABLED=y
CONFIG_ESP_HOSTED_SPI_HOST_INTERFACE=y      # SPI transport (SDIO/UART/SPI-HD disabled)
CONFIG_ESP_HOSTED_IDF_SLAVE_TARGET="esp32c6"
CONFIG_ESP_HOSTED_SPI_MODE=3                # Mode 3 (CPOL=1, CPHA=1) for non-ESP32 slave
CONFIG_ESP_HOSTED_SPI_FREQ_ESP32C6=10      # 10 MHz — do NOT raise
CONFIG_ESP_HOSTED_SPI_RESET_ACTIVE_LOW=y    # XIAO EN is active-low
```

 **Do not raise the SPI clock above 10 MHz.** The XIAO slave pins are routed through the ESP32-C6 **GPIO matrix** (not the dedicated IOMUX SPI pins), which adds enough input delay that above ~10 MHz the slave can no longer reliably sample MOSI. 20 MHz was tried first and failed for this reason.

Requirements & behaviour

- The XIAO C6 must be flashed with the matching **esp-hosted SPI slave** firmware (v2.12.9 range) **before** WiFi will come up.
- If the XIAO is missing, unflashed, or mis-wired, `wifi_stack_init()` fails **gracefully**: WiFi is disabled for that boot and the welder UI keeps running (see [wifi_bridge.cpp](#) (`../ESP32P4/main/wifi_bridge.cpp`) — the C6 radio bring-up is intentionally non-fatal).

See also

- [ESP32P4/SPI_WIFI_INTEGRATION_CHECKLIST.md](#) (`../ESP32P4/SPI_WIFI_INTEGRATION_CHECKLIST.md`) — step-by-step wiring / flashing / boot-log verification and troubleshooting.
- [docs/hardware/PIN_ASSIGNMENTS.md](#) (`PIN_ASSIGNMENTS.md`) — STM32-side pin map.